

Reviewer Pack — Minimal Rank of Tropicalized Cohomology Groups over Boolean ...

Entry 0a8e2dff295b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 07:25:57 UTC

Minimal Rank of Tropicalized Cohomology Groups over Boolean Algebras vs k-CNF Communication Complexity

Entry ID: 0a8e2dff295b **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 07:25:57 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry (Cohomology Groups) **Field B** (complexity object): Communication Complexity: k-CNF

Statement:

[‘For every k-CNF instance with n variables, the minimal rank of its tropicalized cohomology groups over the Boolean algebra is upper-bounded by a function $f(n)$ that is linear in n .’, ‘Equivalently, for all k-CNF instances, the communication complexity is at most $O(f(n))$ bits.’]

Rationale (proposer’s reasoning):

[‘Tropical geometry provides a framework to study complex structures in a simpler form. By applying tropicalization to cohomology groups, we can potentially simplify the complexities associated with computational problems.’, ‘Cohomology groups are a fundamental algebraic invariant that captures topological information. Their minimal rank over Boolean algebras might reveal non-trivial properties of k-CNF instances that are relevant to communication complexity.’]

Taxonomy category: TROPICAL_GEOMETRY_COHOMOLOGY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 3fc0d1981645a81e

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across a statistically significant sample of randomly generated k-CNF instances with n variables, the minimal rank of tropicalized cohomology groups is found to be $\leq f(n)$, where $f(n)$ is linear in n and has a mean value that does not exceed 3 standard deviations above its median. The conjecture is falsified if any seed generates an instance with a minimal rank exceeding $f(n)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 8 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropical geometry cohomology groups AND k-CNF communication complexity
- minimal rank tropicalized cohomology Boolean algebras AND k-CNF complexity - communication complexity upper bound linear function n AND tropical geometry cohomology

Top relevant hits considered: - [http://arxiv.org/abs/2106.11479v4] Differential forms and cohomology in tropical and complex geometry - [http://arxiv.org/abs/1207.2443v2] Tropical Teichmuller and Siegel spaces - [http://arxiv.org/abs/1207.1925v1] Introduction to tropical algebraic geometry - [http://arxiv.org/abs/math/0107200v3] Cohomology of trivial extensions of Frobenius algebras - [http://arxiv.org/abs/cs/0512008v2] Cohomology in Grothendieck Topologies and Lower Bounds in Boolean Complexity - [http://arxiv.org/abs/2512.16347v3] GWTC-4.0: Searches for Gravitational-Wave Lensing Signatures - [http://arxiv.org/abs/1709.08594v2] Upper bounds on Betti numbers of tropical prevarieties - [http://arxiv.org/abs/1306.3497v2] The number of vertices of a tropical curve is bounded by its area

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        cnf = []
        for _ in range(k):
            clause = set(random.sample(range(1, n+1), 2))
            cnf.append(clause)
        return cnf

    def tropicalized_cohomology(cnf):
        # Placeholder function to simulate the computation
        cohomology = {}
        max_var = 0
        for clause in cnf:
            for var in clause:
                if var > max_var:
```

```

        max_var = var
        if var not in cohomology or len(clause) > len(cohomology[var]):
            cohomology[var] = len(clause)
        return cohomology, max_var

def minimal_rank(cohomology):
    return sum(len(v) for v in cohomology.values())

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    total_rank = 0
    for _ in range(5): # Ensure at least 30 instances per seed
        cnf = generate_k_cnf(n, random.randint(1, n))
        cohomology, max_var = tropicalized_cohomology(cnf)
        rank = minimal_rank(cohomology)
        total_rank += rank

    avg_rank = total_rank / len(n_values)
    results.append(avg_rank)

metric_value = sum(results) / len(results)
conjecture_holds = all(rank <= 3 * (metric_value - median) for rank in results)
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank",
    "metric_value": metric_value,
    "instances_tested": len(results),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result["metric_value"])

    mean = sum(results) / len(results)
    std = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
    support_fraction = sum(1 for r in results if r <= 3 * (mean - median)) / len(results)

    if all(r <= 3 * (mean - median) for r in results):
        print(f"RESULT: SUPPORTED mean={mean} std={std} support_fraction={support_fraction}")
    elif any(r > 3 * (mean - median) for r in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if result > 3 * (mean - median))
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9801e51e.py", line 75, in <module>
    trial_result = run_trial(seed)
                   ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9801e51e.py", line 50, in run_trial
    cohomology, max_var = tropicalized_cohomology(cnf)
                        ~~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_9801e51e.py", line 36, in tropicalized_
    if var not in cohomology or len clause > len cohomology var :
                                   ~~~~~~
```

TypeError: object of type 'int' has no len()

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide a result. | next: Investigate the cause of the crash and ensure that the test can run to completion. Once stable, re-run the test with a statistically significant sample of randomly generated k-CNF instances to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17066
2	propose	ollama_remote	glm4:latest	0	0	13090
3	preregistration	ollama_remote	glm4:latest	0	0	9471
4	novelty	ollama_remote	glm4:latest	0	0	8455
5	novelty	ollama_remote	glm4:latest	0	0	9226
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18551
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9935
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8654
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8723
10	judge	ollama_remote	glm4:latest	0	0	14768

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117937 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/0a8e2dff295b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0a8e2dff295b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0a8e2dff295b.tar.gz` (if generated)